ONEDRAFT

CAD-AI — Aria Powered

Executable Foundation Implementation Specification

Version 1.0

Status: Implementation Foundation
Architecture State: Persistence/API Contract Established
Database: PostgreSQL
API: REST / JSON
API Version: `/api/v1`
Identifiers: UUID
Primary Model: Versioned OneDraft Project Model
AI Layer: Aria Powered
Implementation Language: Python
Domain Validation: Pydantic
API Contract: OpenAPI 3
Transaction Model: PostgreSQL atomic transactions
Geometry Model: Version-bound asynchronous computation

---

1. IMPLEMENTATION OBJECTIVE

The OneDraft architecture has now progressed beyond conceptual system design.

The database schema, project model, versioning model, regulatory model, validation model, drawing model, redline model, document provenance model, acceptance workflow, and API boundary have been established.

The purpose of this implementation stage is therefore to convert those architectural decisions into executable software artifacts.

The first implementation foundation shall consist of:

```
001_initial_schema.sql
openapi.yaml
domain_types.py
project_model_service.py
```

These artifacts establish the initial executable boundary between:

```
CLIENT
    ↓
OPENAPI CONTRACT
```

```
    ↓
API / APPLICATION SERVICES
    ↓
DOMAIN TYPES
    ↓
PROJECT MODEL SERVICE
    ↓
POSTGRESQL
    ↓
ASYNC WORKERS
    ↓
GEOMETRY / DOCUMENT / VALIDATION ENGINES
```

The implementation must preserve the distinction between the computational project model and the drawings derived from that model.

---

2. IMPLEMENTATION PRINCIPLES

The executable foundation shall preserve the following architectural principles.

The PostgreSQL database is the authoritative transactional persistence layer.

The Project Model is the authoritative representation of the architectural project.

Revisions are immutable historical states.

Model versions identify exact computational states.

Geometry is version-bound derived state.

Drawings are derived documentation.

Document packages are reproducible outputs.

Code manifests identify the regulatory inputs used for validation.

Validation results identify the state against which validation was performed.

Aria does not directly manipulate PostgreSQL.

Aria does not directly replace files.

Aria does not directly modify production configuration.

Aria produces structured commands that pass through the Project Model command boundary.

Every mutation must be authorized, validated, versioned, and auditable.

---

3. IMPLEMENTATION REPOSITORY STRUCTURE

The initial repository shall contain:

```
OneDraft/
├── README.md
├── LICENSE
├── pyproject.toml
├── .env.example
├── docker-compose.yml
│
├── apps/
│   ├── web/
│   ├── api/
│   └── worker/
│
├── packages/
│   ├── domain/
│   ├── geometry/
│   ├── compliance/
│   ├── documentation/
│   └── infrastructure/
│
├── db/
│   └── migrations/
│       └── 001_initial_schema.sql
│
├── docs/
│   └── openapi/
│       └── openapi.yaml
│
├── packages/domain/
│   └── domain_types.py
│
├── packages/domain/
│   └── project_model_service.py
│
└── tests/
    ├── integration/
    ├── domain/
    ├── api/
    └── fixtures/
```

The exact packaging arrangement may subsequently be refined as implementation proceeds, but the four foundation artifacts remain authoritative implementation outputs.

---

4. ARTIFACT A — 001_initial_schema.sql

The first PostgreSQL migration establishes the complete transactional persistence foundation.

The migration shall be immutable after deployment.

A later correction shall occur through a subsequent migration rather than modification of an already-applied migration.

The migration begins by establishing required extensions.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

UUID generation shall use PostgreSQL-supported UUID generation.

The migration then creates the logical schemas:

```
CREATE SCHEMA IF NOT EXISTS identity;
CREATE SCHEMA IF NOT EXISTS projects;
CREATE SCHEMA IF NOT EXISTS model;
CREATE SCHEMA IF NOT EXISTS geometry;
CREATE SCHEMA IF NOT EXISTS documentation;
CREATE SCHEMA IF NOT EXISTS compliance;
CREATE SCHEMA IF NOT EXISTS validation;
CREATE SCHEMA IF NOT EXISTS revisions;
CREATE SCHEMA IF NOT EXISTS audit;
CREATE SCHEMA IF NOT EXISTS billing;
CREATE SCHEMA IF NOT EXISTS system;
```

---

5. COMMON DATABASE ENUMERATION STRATEGY

The MVP shall prefer controlled VARCHAR status fields over PostgreSQL ENUM types where application evolution is expected to be rapid.

This permits application-level validation while avoiding excessive migration coupling.

Initial status domains include:

```
ACTIVE
INACTIVE
DRAFT
PROPOSED
OPEN
IMPLEMENTED
RESOLVED
QUEUED
RUNNING
VALIDATING
COMPLETED
FAILED
CANCELLED
ACCEPTED
REJECTED
DELETED
ARCHIVED
```

Domain-specific validation shall be enforced by Pydantic and service-layer rules.

Database-level `CHECK` constraints shall be introduced selectively where integrity materially benefits from database enforcement.

---

6. IDENTITY TABLES

The migration shall create:

```
CREATE TABLE identity.organizations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    organization_type VARCHAR(64),
    status VARCHAR(32) NOT NULL,
    billing_account_id UUID,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

The user table:

```
CREATE TABLE identity.users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(320) NOT NULL UNIQUE,
    display_name VARCHAR(255),
    status VARCHAR(32) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Organization membership:

```
CREATE TABLE identity.organization_members (
    organization_id UUID NOT NULL REFERENCES identity.organizations(id),
    user_id UUID NOT NULL REFERENCES identity.users(id),
    role VARCHAR(64) NOT NULL,
    status VARCHAR(32) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (organization_id, user_id)
);
```

Indexes shall be created for organization status, billing references, user membership, and active membership queries.

---

7. PROJECT PERSISTENCE

The project table becomes the principal tenancy boundary:

```
CREATE TABLE projects.projects (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL
```

```
        REFERENCES identity.organizations(id),
    name VARCHAR(255) NOT NULL,
    project_type VARCHAR(64),
    status VARCHAR(32) NOT NULL,
    description TEXT,
    address JSONB,
    jurisdiction JSONB,
    unit_system VARCHAR(32) NOT NULL,
    current_revision_id UUID,
    current_model_version_id UUID,
    current_code_manifest_id UUID,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

The project table must not contain the complete architectural model.

It identifies the project and points toward the current model state.

---

8. PROJECT REQUIREMENTS

```
CREATE TABLE projects.requirements (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    source VARCHAR(64) NOT NULL,
    description TEXT NOT NULL,
    requirement_type VARCHAR(64) NOT NULL,
    priority INTEGER NOT NULL DEFAULT 0,
    mandatory BOOLEAN NOT NULL DEFAULT FALSE,
    status VARCHAR(32) NOT NULL,
    verification_status VARCHAR(32) NOT NULL,
    affected_objects JSONB,
    created_revision UUID,
    modified_revision UUID,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Requirements are project-level inputs to the computational model.

They may originate from:

```
CUSTOMER
ARCHITECT
ENGINEER
CODE
JURISDICTION
SITE
ARIA
SYSTEM
```

---

9. MODEL TABLES

The migration creates the architectural model hierarchy.

```
CREATE TABLE model.buildings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    name VARCHAR(255) NOT NULL,
    building_type VARCHAR(64),
    occupancy VARCHAR(128),
    construction_type VARCHAR(128),
    height NUMERIC,
    area NUMERIC,
    number_of_levels INTEGER,
    geometry_reference_id UUID,
    status VARCHAR(32) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Levels:

```
CREATE TABLE model.levels (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    building_id UUID NOT NULL
        REFERENCES model.buildings(id),
    name VARCHAR(128) NOT NULL,
    level_code VARCHAR(64) NOT NULL,
    elevation NUMERIC NOT NULL,
    floor_to_floor_height NUMERIC,
    finished_floor_elevation NUMERIC,
    ceiling_elevation NUMERIC,
    sequence INTEGER NOT NULL,
    status VARCHAR(32) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(building_id, level_code)
);
```

Spaces:

```
CREATE TABLE model.spaces (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    level_id UUID NOT NULL
        REFERENCES model.levels(id),
    space_number VARCHAR(64),
    name VARCHAR(255),
    space_type VARCHAR(64),
    occupancy VARCHAR(128),
    area NUMERIC,
    volume NUMERIC,
    boundary_geometry_reference_id UUID,
    ceiling_height NUMERIC,
    accessibility_class VARCHAR(64),
    fire_classification VARCHAR(64),
    status VARCHAR(32) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Elements:

```
CREATE TABLE model.elements (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    level_id UUID
        REFERENCES model.levels(id),
    parent_id UUID
        REFERENCES model.elements(id),
    element_type VARCHAR(64) NOT NULL,
    name VARCHAR(255),
    geometry_reference_id UUID,
    assembly_id UUID,
    host_element_id UUID
        REFERENCES model.elements(id),
    parameters JSONB NOT NULL DEFAULT '{}'::jsonb,
    status VARCHAR(32) NOT NULL,
    created_revision_id UUID,
    modified_revision_id UUID,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

The self-referencing `parent_id` permits hierarchical model structures.

The `host_element_id` permits relationships such as:

```
WINDOW → WALL
DOOR → WALL
FINISH → HOST
FIXTURE → SPACE
```

---

10. ASSEMBLIES

```
CREATE TABLE model.assemblies (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    name VARCHAR(255) NOT NULL,
    assembly_type VARCHAR(64),
    layers JSONB NOT NULL,
    total_thickness NUMERIC,
    performance_properties JSONB,
    fire_rating VARCHAR(64),
    thermal_properties JSONB,
    acoustic_properties JSONB,
    manufacturer_data JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Assemblies represent compound building-system definitions rather than individual geometric elements.

---

11. CONSTRAINTS AND RELATIONSHIPS

```
CREATE TABLE model.constraints (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    constraint_type VARCHAR(64) NOT NULL,
    source VARCHAR(64) NOT NULL,
    priority INTEGER NOT NULL DEFAULT 0,
    value JSONB,
    unit VARCHAR(32),
    tolerance JSONB,
    affected_objects JSONB NOT NULL,
    status VARCHAR(32) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Relationships:

```
CREATE TABLE model.relationships (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    source_object_id UUID NOT NULL,
    target_object_id UUID NOT NULL,
    relationship_type VARCHAR(64) NOT NULL,
    metadata JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

The relationship table forms the initial dependency graph of the OneDraft computational model.

---

12. SITE MODEL

```
CREATE TABLE model.sites (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL UNIQUE
        REFERENCES projects.projects(id),
    boundary_geometry JSONB,
    north_angle NUMERIC,
    elevation_reference VARCHAR(128),
    topography JSONB,
    existing_conditions JSONB,
    site_constraints JSONB,
    setbacks JSONB,
    easements JSONB,
    rights_of_way JSONB,
```

```
    utilities JSONB,
    parking JSONB,
    access JSONB,
    survey_reference VARCHAR(1024),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

The site subsystem remains capable of accepting more sophisticated geometry infrastructure later.

---

13. GEOMETRY PERSISTENCE

```
CREATE TABLE geometry.geometry_references (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    object_id UUID NOT NULL,
    geometry_type VARCHAR(32) NOT NULL,
    coordinate_system VARCHAR(128),
    geometry_version INTEGER NOT NULL,
    storage_reference TEXT NOT NULL,
    bounding_box JSONB,
    geometry_hash VARCHAR(128) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(object_id, geometry_version)
);
```

Geometry payloads are not required to reside directly inside the relational model.

The database stores the authoritative reference, version, hash, and provenance.

---

14. MODEL VERSIONING

```
CREATE TABLE revisions.model_versions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    revision_number INTEGER NOT NULL,
    parent_version_id UUID
        REFERENCES revisions.model_versions(id),
    created_by UUID NOT NULL
        REFERENCES identity.users(id),
    state_hash VARCHAR(128) NOT NULL,
    status VARCHAR(32) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(project_id, revision_number)
);
```

A model version represents a precise computational state.

It must be possible to identify the exact model state from:

```
project_id
```

```
model_version_id
state_hash
```

---

15. REVISION RECORDS

```
CREATE TABLE revisions.revisions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    revision_number INTEGER NOT NULL,
    parent_revision_id UUID
        REFERENCES revisions.revisions(id),
    model_version_id UUID NOT NULL
        REFERENCES revisions.model_versions(id),
    reason TEXT,
    created_by UUID NOT NULL
        REFERENCES identity.users(id),
    status VARCHAR(32) NOT NULL,
    code_manifest_id UUID,
    validation_summary JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(project_id, revision_number)
);
```

Revisions are historical records.

A finalized revision is never overwritten.

---

16. CHANGE RECORDS

```
CREATE TABLE revisions.change_records (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    revision_id UUID NOT NULL
        REFERENCES revisions.revisions(id),
    object_id UUID NOT NULL,
    operation VARCHAR(64) NOT NULL,
    before_state JSONB,
    after_state JSONB,
    reason TEXT,
    source VARCHAR(64),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Change records provide the basis for:

```
revision comparison
audit reconstruction
redline verification
AI change explanation
model rollback analysis
```

---

17. ARIA COMMAND PERSISTENCE

```
CREATE TABLE model.aria_commands (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    revision_id UUID
        REFERENCES revisions.revisions(id),
    operation VARCHAR(64) NOT NULL,
    target_objects JSONB NOT NULL,
    parameters JSONB NOT NULL,
    preserve_constraints JSONB,
    reason TEXT,
    confidence NUMERIC,
    requested_by UUID NOT NULL
        REFERENCES identity.users(id),
    validation_requirements JSONB,
    status VARCHAR(32) NOT NULL,
    result JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    completed_at TIMESTAMPTZ
);
```

This is the principal persistence boundary between natural-language Aria intent and executable project-model operations.

---

18. DESIGN PROPOSALS

```
CREATE TABLE model.design_proposals (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    description TEXT NOT NULL,
    affected_objects JSONB,
    proposed_changes JSONB NOT NULL,
    rationale TEXT,
    constraints JSONB,
    estimated_impacts JSONB,
    status VARCHAR(32) NOT NULL,
    created_by UUID NOT NULL
        REFERENCES identity.users(id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Design proposals are distinct from executed commands.

A proposal describes a candidate design state.

A command performs an authorized model mutation.

19. DOCUMENTATION TABLES

Views:

```
CREATE TABLE documentation.views (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    view_type VARCHAR(64) NOT NULL,
    level_id UUID
        REFERENCES model.levels(id),
    section_plane JSONB,
    orientation JSONB,
    scale VARCHAR(64),
    visibility_settings JSONB,
    model_version_id UUID NOT NULL
        REFERENCES revisions.model_versions(id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Drawings:

```
CREATE TABLE documentation.drawings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    view_id UUID NOT NULL
        REFERENCES documentation.views(id),
    drawing_number VARCHAR(64) NOT NULL,
    title VARCHAR(255) NOT NULL,
    discipline VARCHAR(64),
    scale VARCHAR(64),
    model_version_id UUID NOT NULL
        REFERENCES revisions.model_versions(id),
    revision_id UUID NOT NULL
        REFERENCES revisions.revisions(id),
    status VARCHAR(32) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(project_id, drawing_number, revision_id)
);
```

Sheets:

```
CREATE TABLE documentation.sheets (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    sheet_number VARCHAR(64) NOT NULL,
    title VARCHAR(255) NOT NULL,
    discipline VARCHAR(64),
    paper_size VARCHAR(32) NOT NULL,
    template_id UUID,
    revision VARCHAR(32),
```

```
    status VARCHAR(32) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Sheet/drawing association:

```
CREATE TABLE documentation.sheet_drawings (
    sheet_id UUID NOT NULL
        REFERENCES documentation.sheets(id),
    drawing_id UUID NOT NULL
        REFERENCES documentation.drawings(id),
    position JSONB,
    viewport JSONB,
    display_settings JSONB,
    PRIMARY KEY(sheet_id, drawing_id)
);
```

---

20. SHEET TEMPLATES

```
CREATE TABLE documentation.sheet_templates (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    paper_size VARCHAR(32) NOT NULL,
    orientation VARCHAR(32),
    title_block JSONB,
    graphic_standard JSONB,
    revision_block JSONB,
    version INTEGER NOT NULL,
    status VARCHAR(32) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

The initial development seed shall include an ARCH D template.

---

21. ANNOTATION AND DIMENSION MODEL

```
CREATE TABLE documentation.dimensions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    view_id UUID NOT NULL
        REFERENCES documentation.views(id),
    reference_objects JSONB NOT NULL,
    dimension_type VARCHAR(64) NOT NULL,
    value NUMERIC,
    unit VARCHAR(32),
    tolerance JSONB,
    display_style JSONB,
    status VARCHAR(32) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
```

```
);

CREATE TABLE documentation.annotations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    view_id UUID NOT NULL
        REFERENCES documentation.views(id),
    annotation_type VARCHAR(64) NOT NULL,
    text TEXT,
    target_object_id UUID,
    position JSONB,
    style JSONB,
    status VARCHAR(32) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

---

22. SCHEDULE MODEL

```
CREATE TABLE documentation.schedules (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    schedule_type VARCHAR(64) NOT NULL,
    source_objects JSONB NOT NULL,
    columns JSONB NOT NULL,
    sorting JSONB,
    filtering JSONB,
    view_id UUID
        REFERENCES documentation.views(id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Schedules shall be derived from the project model rather than independently maintained copies.

---

23. REDLINE MODEL

```
CREATE TABLE revisions.redlines (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    revision_id UUID NOT NULL
        REFERENCES revisions.revisions(id),
    drawing_id UUID
        REFERENCES documentation.drawings(id),
    target_object_id UUID,
    markup_geometry JSONB,
    instruction TEXT NOT NULL,
    priority VARCHAR(32),
    created_by UUID NOT NULL
        REFERENCES identity.users(id),
```

```
    status VARCHAR(32) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Resolution:

```
CREATE TABLE revisions.redline_resolutions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    redline_id UUID NOT NULL
        REFERENCES revisions.redlines(id),
    command_id UUID
        REFERENCES model.aria_commands(id),
    result TEXT,
    implemented_changes JSONB,
    validation_results JSONB,
    resolved_by UUID
        REFERENCES identity.users(id),
    resolved_at TIMESTAMPTZ
);
```

A redline cannot become `RESOLVED` merely because a response was generated.

Resolution requires an implemented model operation and corresponding verification.

---

24. COMPLIANCE MODEL

Code sources:

```
CREATE TABLE compliance.code_sources (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    jurisdiction VARCHAR(255) NOT NULL,
    authority VARCHAR(255),
    title VARCHAR(512) NOT NULL,
    source_uri TEXT,
    version VARCHAR(128),
    effective_date DATE,
    retrieved_at TIMESTAMPTZ,
    content_hash VARCHAR(128),
    status VARCHAR(32) NOT NULL
);
```

Code rules:

```
CREATE TABLE compliance.code_rules (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    code_source_id UUID NOT NULL
        REFERENCES compliance.code_sources(id),
    rule_identifier VARCHAR(128) NOT NULL,
    citation VARCHAR(255),
    rule_type VARCHAR(64),
    requirement TEXT NOT NULL,
    applicability JSONB,
    threshold JSONB,
```

```
    unit VARCHAR(32),
    exceptions JSONB,
    verification_method JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Code manifests:

```
CREATE TABLE compliance.code_manifests (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    jurisdiction VARCHAR(255) NOT NULL,
    municipality VARCHAR(255),
    province_state VARCHAR(255),
    country VARCHAR(128),
    source_ids JSONB NOT NULL,
    source_versions JSONB NOT NULL,
    effective_dates JSONB,
    retrieval_timestamp TIMESTAMPTZ NOT NULL,
    content_hash VARCHAR(128) NOT NULL,
    status VARCHAR(32) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Once a code manifest is attached to a finalized revision, its contents and provenance become immutable.

---

25. VALIDATION MODEL

```
CREATE TABLE validation.validation_runs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    revision_id UUID NOT NULL
        REFERENCES revisions.revisions(id),
    model_version_id UUID NOT NULL
        REFERENCES revisions.model_versions(id),
    code_manifest_id UUID
        REFERENCES compliance.code_manifests(id),
    started_at TIMESTAMPTZ,
    completed_at TIMESTAMPTZ,
    status VARCHAR(32) NOT NULL,
    result_count INTEGER NOT NULL DEFAULT 0,
    summary JSONB
);
```

Validation results:

```
CREATE TABLE validation.validation_results (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    validation_run_id UUID NOT NULL
        REFERENCES validation.validation_runs(id),
```

```
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    revision_id UUID NOT NULL
        REFERENCES revisions.revisions(id),
    rule_id UUID
        REFERENCES compliance.code_rules(id),
    affected_objects JSONB,
    category VARCHAR(64) NOT NULL,
    status VARCHAR(32) NOT NULL,
    severity VARCHAR(32) NOT NULL,
    description TEXT NOT NULL,
    source JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

---

26. DOCUMENT PACKAGE MODEL

```
CREATE TABLE documentation.document_packages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    revision_id UUID NOT NULL
        REFERENCES revisions.revisions(id),
    model_version_id UUID NOT NULL
        REFERENCES revisions.model_versions(id),
    code_manifest_id UUID
        REFERENCES compliance.code_manifests(id),
    file_references JSONB NOT NULL,
    document_hash VARCHAR(128),
    generated_at TIMESTAMPTZ,
    status VARCHAR(32) NOT NULL
);
```

Every document package must identify the exact model and regulatory state from which it was produced.

---

27. ACCEPTANCE MODEL

```
CREATE TABLE revisions.acceptance_records (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    revision_id UUID NOT NULL
        REFERENCES revisions.revisions(id),
    model_version_id UUID NOT NULL
        REFERENCES revisions.model_versions(id),
    document_package_id UUID NOT NULL
        REFERENCES documentation.document_packages(id),
    code_manifest_id UUID
        REFERENCES compliance.code_manifests(id),
    customer_id UUID NOT NULL
```

```
        REFERENCES identity.users(id),
    accepted_at TIMESTAMPTZ,
    acceptance_status VARCHAR(32) NOT NULL,
    acknowledgements JSONB
);
```

Professional review:

```
CREATE TABLE revisions.professional_reviews (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    revision_id UUID NOT NULL
        REFERENCES revisions.revisions(id),
    reviewer_id UUID NOT NULL
        REFERENCES identity.users(id),
    professional_role VARCHAR(128),
    review_scope TEXT,
    result VARCHAR(64) NOT NULL,
    comments TEXT,
    reviewed_at TIMESTAMPTZ
);
```

---

28. APPROVAL STATUS

```
CREATE TABLE projects.approval_statuses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    authority VARCHAR(255) NOT NULL,
    application_reference VARCHAR(255),
    status VARCHAR(64) NOT NULL,
    submitted_at TIMESTAMPTZ,
    decision_at TIMESTAMPTZ,
    notes TEXT
);
```

Municipal approval is an external administrative state and must not be confused with OneDraft's internal validation state.

---

29. AUDIT EVENT STORE

```
CREATE TABLE audit.project_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL
        REFERENCES projects.projects(id),
    event_type VARCHAR(128) NOT NULL,
    actor_id UUID,
    revision_id UUID,
    object_id UUID,
```

```
    payload JSONB NOT NULL,
    timestamp TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

This table is append-only.

Application services shall never update or delete historical audit events during ordinary operation.

---

30. IDEMPOTENCY STORE

The initial implementation shall include a system-level idempotency table:

```
CREATE TABLE system.idempotency_keys (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    idempotency_key VARCHAR(255) NOT NULL UNIQUE,
    request_hash VARCHAR(128) NOT NULL,
    response_hash VARCHAR(128),
    response_payload JSONB,
    status VARCHAR(32) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    completed_at TIMESTAMPTZ
);
```

Mutating API requests shall use:

```
Idempotency-Key
```

This prevents duplicate mutations caused by client retries.

---

31. DATABASE INDEX FOUNDATION

The initial migration shall establish indexes around principal access patterns:

```
CREATE INDEX idx_projects_organization
ON projects.projects(organization_id);

CREATE INDEX idx_projects_status
ON projects.projects(status);

CREATE INDEX idx_projects_updated
ON projects.projects(updated_at);

CREATE INDEX idx_requirements_project
ON projects.requirements(project_id);

CREATE INDEX idx_buildings_project
ON model.buildings(project_id);

CREATE INDEX idx_levels_building_sequence
ON model.levels(building_id, sequence);
```

```
CREATE INDEX idx_spaces_level
ON model.spaces(level_id);

CREATE INDEX idx_elements_project
ON model.elements(project_id);

CREATE INDEX idx_elements_level
ON model.elements(level_id);

CREATE INDEX idx_elements_parent
ON model.elements(parent_id);

CREATE INDEX idx_elements_type
ON model.elements(element_type);

CREATE INDEX idx_elements_host
ON model.elements(host_element_id);

CREATE INDEX idx_relationships_source
ON model.relationships(source_object_id);

CREATE INDEX idx_relationships_target
ON model.relationships(target_object_id);

CREATE INDEX idx_changes_revision
ON revisions.change_records(revision_id);

CREATE INDEX idx_changes_object
ON revisions.change_records(object_id);

CREATE INDEX idx_aria_commands_project
ON model.aria_commands(project_id);

CREATE INDEX idx_aria_commands_revision
ON model.aria_commands(revision_id);

CREATE INDEX idx_aria_commands_status
ON model.aria_commands(status);

CREATE INDEX idx_validation_project
ON validation.validation_results(project_id);

CREATE INDEX idx_validation_revision
ON validation.validation_results(revision_id);

CREATE INDEX idx_validation_run
ON validation.validation_results(validation_run_id);

CREATE INDEX idx_validation_status
ON validation.validation_results(status);

CREATE INDEX idx_validation_severity
ON validation.validation_results(severity);

CREATE INDEX idx_events_project
ON audit.project_events(project_id);

CREATE INDEX idx_events_timestamp
ON audit.project_events(timestamp);
```

Further indexes shall be added based upon actual query profiles rather than speculative optimization.

---

32. DATABASE MIGRATION COMPLETION

The first migration shall conclude only after all required tables, foreign keys, unique constraints, indexes, and development seed records have been established.

The migration order is:

```
001_extensions
002_identity
003_projects
004_requirements
005_buildings
006_levels
007_sites
008_spaces
009_assemblies
010_elements
011_constraints
012_relationships
013_geometry
014_model_versions
015_revisions
016_commands
017_documentation
018_redlines
019_compliance
020_validation
021_documents
022_acceptance
023_events
024_indexes
025_seed_data
```

For the first executable foundation, these logical stages may be consolidated into `001_initial_schema.sql`, while preserving the dependency order internally.

---

33. ARTIFACT B — OPENAPI.YAML

The OpenAPI contract shall be located at:

```
/docs/openapi/openapi.yaml
```

The specification shall declare:

```
openapi: 3.1.0

info:
  title: OneDraft API
```

```
  version: 0.1.0
  description: OneDraft CAD-AI project modeling and documentation API

servers:
  - url: /api/v1
```

The API shall use bearer authentication:

```
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
```

All protected routes shall require:

```
security:
  - bearerAuth: []
```

---

34. OPENAPI RESOURCE STRUCTURE

The initial API shall expose the following resource families:

```
Organizations
Projects
Requirements
Buildings
Levels
Spaces
Elements
Constraints
Commands
Aria
Revisions
Redlines
Views
Drawings
Sheets
Documents
Compliance
Validation
Exports
Jobs
Acceptance
Events
```

---

35. PROJECT ENDPOINTS

The initial project endpoints are:

```
POST   /projects
GET    /projects
GET    /projects/{project_id}
PATCH  /projects/{project_id}
DELETE /projects/{project_id}
```

Project creation:

```
POST /projects
```

Request:

```
{
  "name": "Clark Residence",
  "project_type": "RESIDENTIAL",
  "description": "Two-storey residence",
  "address": {
    "street": "...",
    "city": "...",
    "province": "...",
    "country": "...",
    "postal_code": "..."
  },
  "unit_system": "METRIC"
}
```

Response:

```
{
  "data": {
    "project_id": "uuid",
    "revision_id": "uuid",
    "model_version_id": "uuid"
  },
  "metadata": {
    "request_id": "uuid",
    "timestamp": "ISO-8601"
  }
}
```

---

36. MODEL ENDPOINTS

Initial model endpoints:

```
POST /projects/{project_id}/buildings
GET  /projects/{project_id}/buildings

POST /buildings/{building_id}/levels
GET  /buildings/{building_id}/levels

POST /levels/{level_id}/spaces
GET  /levels/{level_id}/spaces

POST /projects/{project_id}/elements
```

```
GET  /projects/{project_id}/elements

GET /elements/{element_id}
PATCH /elements/{element_id}
```

All model mutations must operate against a specific current project state.

---

37. ARIA ENDPOINT

The principal Aria interface is:

```
POST /projects/{project_id}/aria/requests
```

Request:

```
{
  "message": "Move the rear bedroom wall 300 mm north while keeping the exterior
envelope unchanged.",
  "revision_id": "uuid",
  "mode": "DESIGN_MODIFICATION"
}
```

The endpoint must not directly modify the model merely because natural-language intent was received.

The request enters the command pipeline.

The response identifies the proposed command:

```
{
  "data": {
    "command_id": "uuid",
    "status": "PROPOSED",
    "operation": "MOVE_ELEMENT",
    "targets": ["uuid"],
    "parameters": {
      "distance": 300,
      "unit": "mm",
      "direction": "NORTH"
    }
  }
}
```

---

38. COMMAND EXECUTION

The command execution interface is:

```
POST /projects/{project_id}/commands
```

Request:

```
{
  "operation": "MOVE_ELEMENT",
  "target_objects": [
    "uuid"
  ],
  "parameters": {
    "distance": 300,
    "unit": "mm",
    "direction": "NORTH"
  },
  "preserve_constraints": [
    "uuid"
  ],
  "expected_revision": 14
}
```

The service must verify:

```
authentication
authorization
project membership
expected revision
target existence
operation validity
constraint compatibility
parameter validity
```

before mutation.

---

39. COMMAND RESPONSE

A successful command returns:

```
{
  "data": {
    "command_id": "uuid",
    "status": "COMPLETED",
    "revision_id": "uuid",
    "model_version_id": "uuid",
    "affected_objects": [
      "uuid",
      "uuid"
    ],
    "affected_drawings": [
      "A101",
      "A201",
      "A401"
    ]
  }
}
```

A command that cannot safely execute returns a structured error without modifying the project state.

---

40. REDLINE ENDPOINTS

The initial redline interface is:

```
POST /projects/{project_id}/redlines
GET  /projects/{project_id}/redlines
GET  /redlines/{redline_id}
POST /redlines/{redline_id}/resolve
```

Request:

```
{
  "drawing_id": "uuid",
  "target_object_id": "uuid",
  "instruction": "Add a 900 mm wide window centered on this wall.",
  "markup_geometry": {},
  "priority": "NORMAL"
}
```

A new redline begins as:

```
OPEN
```

It may become:

```
IMPLEMENTED
```

only after the associated model operation has successfully completed.

---

41. VALIDATION ENDPOINT

```
POST /projects/{project_id}/validation
```

Request:

```
{
  "revision_id": "uuid",
  "code_manifest_id": "uuid",
  "categories": [
    "CODE",
    "SPATIAL",
    "ACCESSIBILITY",
    "DOCUMENT",
    "COORDINATION"
  ]
}
```

Response:

```
{
  "data": {
    "validation_run_id": "uuid",
    "status": "QUEUED"
```

```
  }
}
```

Validation executes asynchronously.

---

42. DRAWING GENERATION

`POST /projects/{project_id}/drawings/generate`

Request:

```
{
  "revision_id": "uuid",
  "drawing_types": [
    "PLAN",
    "ELEVATION",
    "SECTION"
  ],
  "sheet_template": "ARCH_D"
}
```

Response:

```
{
  "data": {
    "job_id": "uuid",
    "status": "QUEUED"
  }
}
```

The drawing worker must bind the resulting drawings to the exact:

```
project_id
revision_id
model_version_id
view_id
```

from which they were generated.

---

43. DOCUMENT GENERATION

`POST /projects/{project_id}/documents/generate`

Request:

```
{
  "revision_id": "uuid",
  "format": "PDF",
  "paper_size": "ARCH_D",
  "include": [
```

```
    "PLANS",
    "ELEVATIONS",
    "SECTIONS",
    "SCHEDULES",
    "NOTES"
  ]
}
```

Response:

```
{
  "data": {
    "job_id": "uuid",
    "status": "QUEUED"
  }
}
```

---

44. JOB MODEL

Expensive operations return a Job ID.

Initial job types include:

```
GEOMETRY_GENERATION
VALIDATION
DRAWING_GENERATION
DOCUMENT_GENERATION
EXPORT
```

Job states:

```
QUEUED
RUNNING
VALIDATING
COMPLETED
FAILED
CANCELLED
```

Endpoint:

```
GET /jobs/{job_id}
```

Response:

```
{
  "data": {
    "job_id": "uuid",
    "type": "DOCUMENT_GENERATION",
    "status": "RUNNING",
    "progress": 72
  }
}
```

---

45. STANDARD API RESPONSE

Every successful response shall follow:

```
{
  "data": {},
  "metadata": {
    "request_id": "uuid",
    "timestamp": "ISO-8601"
  }
}
```

Collection responses:

```
{
  "data": [],
  "metadata": {
    "request_id": "uuid",
    "timestamp": "ISO-8601",
    "page": 1,
    "page_size": 50,
    "total": 100
  }
}
```

---

46. STANDARD API ERROR

Errors shall use:

```
{
  "error": {
    "code": "CONSTRAINT_CONFLICT",
    "message": "Requested operation conflicts with an active project constraint.",
    "object_id": "uuid",
    "resolution": "Review the affected constraint or submit a revised command."
  },
  "metadata": {
    "request_id": "uuid",
    "timestamp": "ISO-8601"
  }
}
```

Initial error codes include:

```
INVALID_REQUEST
UNAUTHORIZED
FORBIDDEN
PROJECT_NOT_FOUND
REVISION_NOT_FOUND
MODEL_VERSION_NOT_FOUND
OBJECT_NOT_FOUND
REVISION_CONFLICT
CONSTRAINT_CONFLICT
INVALID_COMMAND
```

```
COMMAND_REJECTED
VALIDATION_FAILED
JOB_FAILED
IDEMPOTENCY_CONFLICT
```

---

47. API PAGINATION

Collection endpoints shall accept:

```
page
page_size
```

The initial maximum page size is:

```
100
```

Future versions may implement cursor pagination without requiring an immediate API break.

---

48. API FILTERING

Collection endpoints shall support:

```
status
type
created_after
created_before
updated_after
updated_before
```

Element collections additionally support:

```
level_id
element_type
parent_id
```

---

49. ARTIFACT C — DOMAIN_TYPES.PY

The Python domain layer shall use Pydantic models to establish a strongly validated boundary between external API data and internal domain services.

The initial module shall define:

```
from __future__ import annotations

from datetime import datetime
from decimal import Decimal
```

```
from enum import Enum
from typing import Any
from uuid import UUID, uuid4

from pydantic import BaseModel, ConfigDict, Field


class UnitSystem(str, Enum):
    METRIC = "METRIC"
    IMPERIAL = "IMPERIAL"


class ProjectStatus(str, Enum):
    DRAFT = "DRAFT"
    ACTIVE = "ACTIVE"
    ARCHIVED = "ARCHIVED"
    DELETED = "DELETED"


class CommandStatus(str, Enum):
    PROPOSED = "PROPOSED"
    QUEUED = "QUEUED"
    RUNNING = "RUNNING"
    COMPLETED = "COMPLETED"
    FAILED = "FAILED"
    CANCELLED = "CANCELLED"


class RevisionStatus(str, Enum):
    DRAFT = "DRAFT"
    ACTIVE = "ACTIVE"
    FINALIZED = "FINALIZED"
    ARCHIVED = "ARCHIVED"


class ProjectCreate(BaseModel):
    model_config = ConfigDict(extra="forbid")

    name: str = Field(min_length=1, max_length=255)
    project_type: str | None = None
    description: str | None = None
    address: dict[str, Any] | None = None
    unit_system: UnitSystem = UnitSystem.METRIC
```

The project representation:

```
class Project(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: UUID
    organization_id: UUID
    name: str
    project_type: str | None
    status: ProjectStatus
    description: str | None
    address: dict[str, Any] | None
    jurisdiction: dict[str, Any] | None
    unit_system: UnitSystem
    current_revision_id: UUID | None
```

```
    current_model_version_id: UUID | None
    current_code_manifest_id: UUID | None
    created_at: datetime
    updated_at: datetime
```

---

50. DOMAIN COMMAND TYPES

The command boundary begins with:

```
class CommandRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")

    operation: str
    target_objects: list[UUID] = Field(default_factory=list)
    parameters: dict[str, Any] = Field(default_factory=dict)
    preserve_constraints: list[UUID] = Field(default_factory=list)
    expected_revision: int
```

The result:

```
class CommandResult(BaseModel):
    command_id: UUID
    status: CommandStatus
    revision_id: UUID | None = None
    model_version_id: UUID | None = None
    affected_objects: list[UUID] = Field(default_factory=list)
    affected_drawings: list[str] = Field(default_factory=list)
    result: dict[str, Any] = Field(default_factory=dict)
```

---

51. ARIA REQUEST TYPE

```
class AriaRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")

    message: str = Field(min_length=1)
    revision_id: UUID
    mode: str = "DESIGN_MODIFICATION"
```

The Aria request is intentionally distinct from a command.

The pipeline is:

```
NATURAL LANGUAGE
        ↓
ARIA REQUEST
        ↓
INTERPRETATION
        ↓
STRUCTURED COMMAND
        ↓
```

```
DOMAIN VALIDATION
        ↓
MODEL MUTATION
```

---

52. MODEL OBJECT TYPES

The first domain implementation shall include:

```
class BuildingCreate(BaseModel):
    name: str
    building_type: str | None = None
    occupancy: str | None = None
    construction_type: str | None = None
    height: Decimal | None = None
    area: Decimal | None = None
    number_of_levels: int | None = None

class LevelCreate(BaseModel):
    name: str
    level_code: str
    elevation: Decimal
    floor_to_floor_height: Decimal | None = None
    finished_floor_elevation: Decimal | None = None
    ceiling_elevation: Decimal | None = None
    sequence: int

class SpaceCreate(BaseModel):
    space_number: str | None = None
    name: str | None = None
    space_type: str | None = None
    occupancy: str | None = None
    area: Decimal | None = None
    volume: Decimal | None = None
    ceiling_height: Decimal | None = None
```

---

53. REDLINE DOMAIN TYPES

```
class RedlineCreate(BaseModel):
    drawing_id: UUID | None = None
    target_object_id: UUID | None = None
    markup_geometry: dict[str, Any] | None = None
    instruction: str = Field(min_length=1)
    priority: str = "NORMAL"

class RedlineResolution(BaseModel):
    redline_id: UUID
    command_id: UUID | None = None
    result: str
    implemented_changes: dict[str, Any] = Field(default_factory=dict)
    validation_results: dict[str, Any] = Field(default_factory=dict)
```

---

54. VALIDATION DOMAIN TYPES

```
class ValidationRequest(BaseModel):
    revision_id: UUID
    code_manifest_id: UUID | None = None
    categories: list[str]

class ValidationRun(BaseModel):
    id: UUID
    project_id: UUID
    revision_id: UUID
    model_version_id: UUID
    code_manifest_id: UUID | None
    status: str
    result_count: int = 0
    summary: dict[str, Any] = Field(default_factory=dict)
```

---

55. DOCUMENT GENERATION TYPES

```
class DrawingGenerationRequest(BaseModel):
    revision_id: UUID
    drawing_types: list[str]
    sheet_template: str = "ARCH_D"

class DocumentGenerationRequest(BaseModel):
    revision_id: UUID
    format: str = "PDF"
    paper_size: str = "ARCH_D"
    include: list[str]
```

---

56. ARTIFACT D — PROJECT_MODEL_SERVICE.PY

The Project Model Service is the first executable domain boundary.

It is responsible for coordinating transactional model mutations.

Its primary responsibilities are:

```
load current project state
verify authorization context
verify revision
verify command
verify constraints
execute mutation
```

```
create new model version
create revision
record change records
record audit event
update current project pointers
return command result
```

It must not contain:

```
HTTP-specific code
OpenAPI-specific code
AI provider-specific code
PDF rendering code
database migration code
UI logic
```

---

57. SERVICE INTERFACE

The service shall expose an interface conceptually equivalent to:

```
class ProjectModelService:
    def execute_command(
        self,
        *,
        project_id: UUID,
        actor_id: UUID,
        command: CommandRequest,
    ) -> CommandResult:
        ...
```

The implementation shall receive a database session or transaction context through dependency injection.

---

58. COMMAND EXECUTION PIPELINE

The command service executes:

```
1. BEGIN TRANSACTION

2. LOAD PROJECT

3. VERIFY ACTOR

4. VERIFY PROJECT MEMBERSHIP

5. VERIFY EXPECTED REVISION

6. LOAD TARGET OBJECTS

7. VALIDATE COMMAND
```

```
8. LOAD ACTIVE CONSTRAINTS

9. CHECK CONSTRAINT COMPATIBILITY

10. APPLY MODEL MUTATION

11. CREATE MODEL VERSION

12. CREATE REVISION

13. CREATE CHANGE RECORDS

14. UPDATE PROJECT CURRENT STATE

15. RECORD ARIA COMMAND RESULT

16. RECORD AUDIT EVENT

17. COMMIT

18. RETURN RESULT
```

Any failure before commit causes:

```
ROLLBACK
```

No partially mutated project may remain.

---

59. REVISION CONCURRENCY CONTROL

The expected revision supplied by the client or Aria command is mandatory for mutating commands.

If the project currently contains:

```
Revision 14
```

and the request expects:

```
Revision 13
```

the service must reject the operation.

The error is:

```
REVISION_CONFLICT
```

This prevents stale clients from silently overwriting newer model state.

---

60. COMMAND VALIDATION

The service shall validate the command operation before mutation.

Initial operations include:

```
CREATE_BUILDING
CREATE_LEVEL
CREATE_SPACE
CREATE_ELEMENT
UPDATE_ELEMENT
MOVE_ELEMENT
DELETE_ELEMENT
CREATE_CONSTRAINT
UPDATE_CONSTRAINT
```

Additional operations shall be introduced through explicit domain implementations.

An unknown operation must fail before any database mutation.

---

61. MOVE_ELEMENT OPERATION

The first geometric command implementation shall establish the `MOVE_ELEMENT` pattern.

Input:

```
{
  "operation": "MOVE_ELEMENT",
  "target_objects": ["uuid"],
  "parameters": {
    "distance": 300,
    "unit": "mm",
    "direction": "NORTH"
  },
  "expected_revision": 14
}
```

The service must determine:

```
target geometry
host relationships
dependent objects
active constraints
affected views
affected drawings
```

The command must preserve all constraints explicitly identified as protected.

---

62. MODEL MUTATION RESULT

A successful mutation creates:

```
new model version
new revision
change records
audit event
command result
```

The project current pointers become:

```
current_revision_id
current_model_version_id
```

and are updated atomically.

---

63. DRAWING IMPACT ANALYSIS

The command service shall identify affected documentation.

For example:

```
MOVE WALL
    ↓
affected wall
    ↓
affected room boundaries
    ↓
affected dimensions
    ↓
affected plan view
    ↓
affected section/elevation views
    ↓
affected drawings
```

The service need not regenerate drawings synchronously.

It must identify the affected documentation so the worker subsystem can regenerate the appropriate outputs.

---

64. GEOMETRY WORKER CONTRACT

The geometry worker receives:

```
{
  "project_id": "uuid",
  "model_version_id": "uuid",
```

```
  "element_id": "uuid",
  "geometry_version": 3
}
```

The worker must verify that the model version remains valid before committing geometry results.

If the project has advanced beyond the referenced model version, the geometry result becomes stale.

A stale result must never overwrite current geometry.

---

65. DRAWING WORKER CONTRACT

Drawing jobs contain:

```
{
  "project_id": "uuid",
  "revision_id": "uuid",
  "model_version_id": "uuid",
  "view_id": "uuid"
}
```

The drawing worker must preserve these provenance identifiers in the resulting documentation record.

---

66. DOCUMENT WORKER CONTRACT

Document generation receives:

```
{
  "project_id": "uuid",
  "revision_id": "uuid",
  "model_version_id": "uuid",
  "code_manifest_id": "uuid",
  "validation_run_id": "uuid"
}
```

The resulting package must retain this complete provenance chain.

---

67. SYSTEM OF RECORD

The implementation shall enforce the following hierarchy:

```
PROJECT MODEL
      ↓
MODEL VERSION
      ↓
REVISION
      ↓
```

```
CODE MANIFEST
      ↓
VALIDATION
      ↓
DRAWINGS
      ↓
DOCUMENT PACKAGE
      ↓
CUSTOMER ACCEPTANCE
```

Audit events exist alongside this chain as the immutable event record.

The drawing is therefore never treated as the authoritative architectural model.

---

68. RECONSTRUCTION SERVICE

A future reconstruction service shall accept:

```
project_id
revision_id
model_version_id
code_manifest_id
```

and reconstruct the exact project state associated with those identifiers.

The reconstruction process shall be capable of determining:

```
project requirements
site
buildings
levels
spaces
elements
assemblies
constraints
relationships
geometry references
views
drawings
validation results
document provenance
```

This is a fundamental acceptance criterion for the architecture.

---

69. FIRST API INTEGRATION

The API layer shall translate HTTP requests into domain commands.

The architecture becomes:

```
HTTP REQUEST
```

```
    ↓
AUTHENTICATION
    ↓
AUTHORIZATION
    ↓
REQUEST VALIDATION
    ↓
DOMAIN COMMAND
    ↓
PROJECT MODEL SERVICE
    ↓
DATABASE TRANSACTION
    ↓
DOMAIN RESULT
    ↓
API RESPONSE
```

The API layer must not directly perform arbitrary SQL mutations.

---

70. ARIA INTEGRATION BOUNDARY

Aria operates above the Project Model Service.

The relationship is:

```
USER
 ↓
ARIA
 ↓
INTENT INTERPRETATION
 ↓
STRUCTURED COMMAND
 ↓
PROJECT MODEL SERVICE
 ↓
VALIDATION
 ↓
MODEL
```

Aria is therefore an intelligence/orchestration layer.

The Project Model Service is the enforcement layer.

The database is the persistence layer.

This separation prevents natural-language interpretation from becoming an uncontrolled database mutation mechanism.

---

71. AI PROVIDER ADAPTER

The underlying model provider remains behind an adapter:

```
Aria Orchestrator
        ↓
AI Provider Adapter
        ↓
Model Provider
```

The public OneDraft API does not expose the underlying provider implementation.

Changing the model provider must not require changing the OneDraft project API.

---

72. API AUTHORIZATION

Every project operation must evaluate:

```
authenticated identity
+
organization membership
+
project membership
+
requested permission scope
```

Initial permissions include:

```
project:read
project:write
model:read
model:write
aria:execute
aria:approve
drawing:read
drawing:generate
redline:read
redline:write
compliance:read
compliance:execute
validation:read
validation:execute
document:read
document:generate
acceptance:write
admin:organization
admin:project
```

---

73. TRANSACTIONAL GUARANTEE

The central invariant is:

```
NO PARTIAL MODEL MUTATION
```

A successful command creates a complete state transition.

A failed command creates no new current state.

The conceptual transaction remains:

```
BEGIN
 ↓
VERIFY
 ↓
MUTATE
 ↓
VERSION
 ↓
RECORD
 ↓
COMMIT
```

or:

```
BEGIN
 ↓
FAIL
 ↓
ROLLBACK
```

---

74. IDEMPOTENT COMMAND EXECUTION

Mutating API requests shall support:

```
Idempotency-Key
```

The server calculates:

```
request_hash
```

If the same key is submitted with an identical request:

```
return previous response
```

If the same key is submitted with a different request:

```
IDEMPOTENCY_CONFLICT
```

This prevents accidental duplicate mutations.

---

75. SEED DATA

The initial development database shall contain:

```
ARCH_D sheet template
basic wall assembly
basic door
basic window
residential project type
commercial project type
initial element types
initial command types
test jurisdiction
```

The test jurisdiction must be explicitly identified as development data.

Production regulatory sources shall never be represented by test fixtures.

---

76. INITIAL INTEGRATION TEST

The first complete OneDraft integration test shall execute:

```
CREATE ORGANIZATION
        ↓
CREATE USER
        ↓
CREATE PROJECT
        ↓
CREATE REQUIREMENTS
        ↓
CREATE BUILDING
        ↓
CREATE GROUND LEVEL
        ↓
CREATE LEVEL 1
        ↓
CREATE ROOMS
        ↓
CREATE WALLS
        ↓
CREATE DOORS
        ↓
CREATE WINDOWS
        ↓
GENERATE PLANS
        ↓
GENERATE ELEVATIONS
        ↓
GENERATE SECTION
        ↓
GENERATE ARCH D SHEETS
        ↓
SUBMIT ARIA MODIFICATION
        ↓
```

```
VALIDATE COMMAND
        ↓
CREATE REVISION 2
        ↓
REGENERATE AFFECTED DRAWINGS
        ↓
CREATE REDLINE
        ↓
RESOLVE REDLINE
        ↓
RUN VALIDATION
        ↓
GENERATE DOCUMENT PACKAGE
        ↓
HASH DOCUMENT PACKAGE
        ↓
RECORD CUSTOMER ACCEPTANCE
        ↓
RETRIEVE COMPLETE PROJECT HISTORY
```

This becomes the first major OneDraft regression test.

---

77. RECONSTRUCTION TEST

The integration suite must additionally perform:

```
retrieve Project_ID
retrieve Revision_ID
retrieve Model_Version_ID
retrieve Code_Manifest_ID
reconstruct project state
verify state hash
verify associated documentation
verify validation provenance
verify document package hash
```

The reconstruction must reproduce the same computational state represented by the stored model version.

---

78. DOCUMENT HASH TEST

When a document package is generated:

```
DOCUMENT CONTENT
       ↓
HASH
       ↓
document_hash
```

The hash is stored with:

```
project_id
revision_id
model_version_id
code_manifest_id
validation_run_id
```

This permits later verification that the package corresponds to the recorded state.

---

79. MIGRATION TEST

The database foundation passes its migration test when:

```
DROP DEVELOPMENT DATABASE
        ↓
CREATE DATABASE
        ↓
RUN 001_INITIAL_SCHEMA
        ↓
VERIFY ALL TABLES
        ↓
VERIFY FOREIGN KEYS
        ↓
VERIFY UNIQUE CONSTRAINTS
        ↓
VERIFY INDEXES
        ↓
VERIFY SEED DATA
```

The migration must be repeatable in a clean development environment.

---

80. API CONTRACT TEST

The OpenAPI contract passes its initial test when:

```
openapi.yaml
        ↓
schema validation
        ↓
operation validation
        ↓
request model validation
        ↓
response model validation
```

No undocumented mutation endpoint may bypass the domain service.

---

81. DOMAIN SERVICE TEST

The Project Model Service must pass tests for:

```
valid command
invalid command
missing project
missing object
invalid revision
stale revision
constraint conflict
unauthorized actor
invalid parameter
successful mutation
rollback
revision creation
model version creation
change record creation
audit event creation
```

---

82. ARIA COMMAND TEST

The first Aria command test shall use:

```
Move the rear bedroom wall 300 mm north while keeping
the exterior envelope unchanged.
```

The expected flow is:

```
ARIA REQUEST
 ↓
STRUCTURED COMMAND
 ↓
TARGET IDENTIFICATION
 ↓
CONSTRAINT IDENTIFICATION
 ↓
COMMAND VALIDATION
 ↓
MODEL MUTATION
 ↓
REVISION CREATION
 ↓
AFFECTED DRAWING IDENTIFICATION
 ↓
SUCCESS
```

The natural-language request itself must never directly become SQL.

---

83. REDLINE TEST

The first redline regression shall:

```
create drawing
create redline
verify OPEN
interpret instruction
create command
execute command
create revision
validate result
record resolution
verify IMPLEMENTED/RESOLVED state
regenerate affected drawing
```

The redline is not complete until the computational model and derived documentation have been updated.

---

84. VALIDATION TEST

The first validation test shall establish:

```
Revision N
    ↓
Code Manifest N
    ↓
Validation Run N
    ↓
Validation Results N
```

The test shall verify that validation results remain permanently associated with the exact revision and regulatory manifest against which they were generated.

---

85. DOCUMENTATION TEST

The first documentation test shall establish:

```
Model Version
    ↓
Revision
    ↓
Views
    ↓
Drawings
    ↓
Sheets
    ↓
Document Package
```

The generated package must never be attributed to a different revision simply because a newer revision later becomes current.

---

86. CUSTOMER ACCEPTANCE TEST

Acceptance shall require:

```
Project
+
Revision
+
Model Version
+
Document Package
+
Code Manifest
```

The acceptance record shall identify the exact package accepted.

Acceptance therefore does not modify historical model state.

---

87. IMPLEMENTATION COMPLETION CRITERIA

The executable foundation is complete when the following are operational:

```
PostgreSQL migration
OpenAPI contract
Pydantic domain models
Project Model Service
transaction boundary
revision creation
model-version creation
change recording
audit recording
idempotency
authorization boundary
Aria command boundary
redline persistence
validation persistence
drawing persistence
document persistence
acceptance persistence
```

---

88. IMPLEMENTATION SEQUENCE

The actual engineering sequence shall be:

```
PHASE 1
PostgreSQL
        ↓
001_initial_schema.sql

PHASE 2
Domain types
        ↓
domain_types.py

PHASE 3
Project Model Service
        ↓
project_model_service.py

PHASE 4
API implementation
        ↓
OpenAPI-conforming endpoints

PHASE 5
Worker infrastructure
        ↓
geometry
validation
drawing
document

PHASE 6
Aria orchestration
        ↓
natural-language → structured command

PHASE 7
Frontend
        ↓
project workspace
model viewer
drawing viewer
redline interface

PHASE 8
Integration test
        ↓
complete project lifecycle
```

---

89. ARCHITECTURAL INVARIANTS

The following invariants are permanent unless explicitly changed through a future architecture revision.

A drawing is not the project model.

A PDF is not the project model.

An AI response is not a project mutation.

A proposed command is not an executed command.

A revision is not overwritten.

A finalized code manifest is not silently changed.

A validation result belongs to the exact model and regulatory state against which it was generated.

A geometry result belongs to the model version that produced it.

A drawing belongs to the revision and model version from which it was generated.

A document package belongs to its exact revision, model version, regulatory manifest, and validation state.

Customer acceptance belongs to a specific document package.

Audit events are append-only.

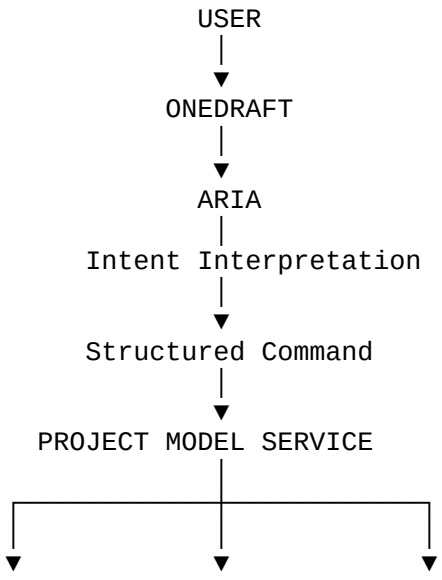
External clients do not connect directly to PostgreSQL.

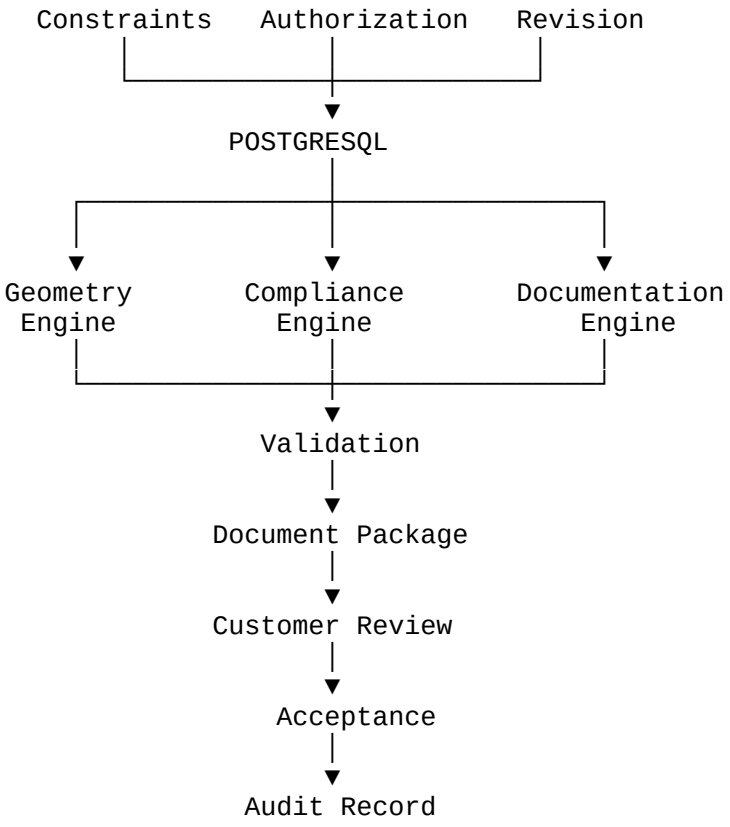
Aria does not bypass the domain service.

---

90. ONEDRAFT EXECUTABLE FOUNDATION

The resulting architecture is:





---

91. FIRST SOFTWARE MILESTONE

The first actual OneDraft software milestone is therefore defined as:

```
CREATE PROJECT
        ↓
CREATE VERSIONED MODEL
        ↓
EXECUTE STRUCTURED COMMAND
        ↓
CREATE NEW REVISION
        ↓
PERSIST CHANGE RECORD
        ↓
PERSIST AUDIT EVENT
        ↓
RETRIEVE CURRENT MODEL
        ↓
RECONSTRUCT PRIOR MODEL
```

This milestone establishes the core computational state machine before sophisticated CAD generation is introduced.

---

92. FIRST DRAWING MILESTONE

Once the Project Model Service is operational, the first drawing milestone is:

```
MODEL
 ↓
PLAN VIEW
 ↓
ELEVATION VIEW
 ↓
SECTION VIEW
 ↓
DIMENSIONS
 ↓
ANNOTATIONS
 ↓
ARCH D SHEET
```

The drawing engine consumes the model.

It does not become the model.

---

93. FIRST REGULATORY MILESTONE

The first regulatory milestone is:

```
PROJECT JURISDICTION
        ↓
CODE SOURCES
        ↓
CODE RULES
        ↓
CODE MANIFEST
        ↓
MODEL APPLICABILITY
        ↓
VALIDATION
        ↓
RESULTS
```

This ensures jurisdiction-specific code analysis is performed against identified regulatory inputs before final documentation.

---

94. FIRST CUSTOMER MILESTONE

The first customer-complete lifecycle is:

```
SPECIFICATION
        ↓
PROJECT MODEL
```

```
        ↓
ARIA DESIGN ITERATION
        ↓
REDLINES
        ↓
VALIDATION
        ↓
DRAWINGS
        ↓
DOCUMENT PACKAGE
        ↓
CUSTOMER REVIEW
        ↓
ACCEPTANCE
```

The lifecycle is persistent and reconstructable.

---

95. IMPLEMENTATION STATE

The OneDraft architecture has now transitioned from conceptual specification into an executable implementation plan.

The established layers are:

```
PRODUCT DEFINITION
SYSTEM ARCHITECTURE
TECHNOLOGY ARCHITECTURE
DOMAIN MODEL
DATABASE SCHEMA
API CONTRACT
VERSIONING MODEL
AI COMMAND BOUNDARY
GEOMETRY BOUNDARY
REGULATORY MODEL
VALIDATION MODEL
DRAWING MODEL
REDLINE MODEL
DOCUMENT MODEL
ACCEPTANCE MODEL
AUDIT MODEL
```

The executable foundation is:

```
001_initial_schema.sql
openapi.yaml
domain_types.py
project_model_service.py
```

---

96. FINAL IMPLEMENTATION DECISION

OneDraft shall be implemented as a computational architectural project system.

It is not fundamentally:

```
CHATBOT
+
IMAGE GENERATOR
+
PDF EXPORT
```

It is:

```
COMPUTATIONAL PROJECT MODEL
+
ARIA INTELLIGENCE
+
GEOMETRY ENGINE
+
REGULATORY ENGINE
+
VALIDATION ENGINE
+
DOCUMENT ENGINE
+
VERSION CONTROL
+
REDLINE WORKFLOW
+
CUSTOMER REVIEW
+
AUDITABLE PROVENANCE
```

The architectural drawing is a derived representation of the computational project model.

The document package is a derived representation of the validated documentation state.

The customer acceptance record is a record of acceptance of a specific derived package.

The audit system preserves what happened.

The revision system preserves what changed.

The model-version system preserves what the project actually was at a particular point in time.

This distinction remains the fundamental architectural principle of OneDraft.

---

97. VERSION 0.1 FOUNDATION STATUS

ONEDRAFT — EXECUTABLE FOUNDATION IMPLEMENTATION SPECIFICATION v0.1

STATUS: ESTABLISHED FOR IMPLEMENTATION

The preceding persistence/API architecture is considered complete at the specification level.

The immediate executable artifacts are:

```
001_initial_schema.sql
openapi.yaml
domain_types.py
project_model_service.py
```

The first engineering objective is to make those artifacts execute together as one coherent transactional system.

The first successful system state is therefore:

```
POSTGRESQL
        +
DOMAIN TYPES
        +
PROJECT MODEL SERVICE
        +
OPENAPI CONTRACT
        ↓
EXECUTABLE ONEDRAFT CORE
```

From that point, geometry generation, regulatory validation, CAD documentation, redline iteration, document packaging, and Aria orchestration become successive computational layers operating against the same persistent project model.

This is the OneDraft software skeleton.